

4B. Deep CNN ensemble model_Stacking

March 7, 2026

Author : Vaibhav Thakur

```
[1]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[2]: import warnings
warnings.filterwarnings("ignore")

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import (Conv2D, MaxPooling2D, Flatten, Dense,
                                     Dropout, BatchNormalization, Input,
                                     Concatenate)

# Define the paths
train_dir = '/content/drive/MyDrive/Deep Learning - datasets/alzheimer dataset/'
↳train'
test_dir = '/content/drive/MyDrive/Deep Learning - datasets/alzheimer dataset/'
↳test'

# Data augmentation for training
train_datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.4,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Simple rescaling for testing/validation
test_datagen = ImageDataGenerator(rescale=1./255)

# Load the images and labels, ensuring grayscale (1 channel)
train_generator = train_datagen.flow_from_directory(
    train_dir,
```

```

        target_size=(128, 128),
        color_mode='grayscale',
        class_mode='categorical'
    )

    test_generator = test_datagen.flow_from_directory(
        test_dir,
        target_size=(128, 128),
        color_mode='grayscale',
        class_mode='categorical'
    )

    # Infer number of classes from the generator
    n_classes = 4
    input_shape = (128, 128, 1)

```

Found 4647 images belonging to 4 classes.

Found 1232 images belonging to 4 classes.

BUILD 3 CNN MODELS

```

[5]: # CNN 1

CNN1 = Sequential()
CNN1.add(Conv2D(256, (3, 3), activation='relu', input_shape=input_shape))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(64, (3, 3), activation='relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(256, (3, 3), activation='relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(128, (3, 3), activation='relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(256, (3, 3), activation='relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Flatten())
CNN1.add(Dense(128, activation='relu'))
CNN1.add(Dropout(0.2))
CNN1.add(Dense(64, activation='relu'))
CNN1.add(Dense(n_classes, activation='softmax'))

CNN1.compile(
    optimizer='adam',
    loss='categorical_crossentropy',

```

```

        metrics=['accuracy']
    )

CNN1.summary()

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 126, 126, 256)	2,560
max_pooling2d_10 (MaxPooling2D)	(None, 63, 63, 256)	0
conv2d_11 (Conv2D)	(None, 61, 61, 64)	147,520
max_pooling2d_11 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_12 (Conv2D)	(None, 28, 28, 256)	147,712
max_pooling2d_12 (MaxPooling2D)	(None, 14, 14, 256)	0
conv2d_13 (Conv2D)	(None, 12, 12, 128)	295,040
max_pooling2d_13 (MaxPooling2D)	(None, 6, 6, 128)	0
conv2d_14 (Conv2D)	(None, 4, 4, 256)	295,168
max_pooling2d_14 (MaxPooling2D)	(None, 2, 2, 256)	0
flatten_2 (Flatten)	(None, 1024)	0
dense_6 (Dense)	(None, 128)	131,200
dropout_2 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8,256
dense_8 (Dense)	(None, 4)	260

Total params: 1,027,716 (3.92 MB)

Trainable params: 1,027,716 (3.92 MB)

Non-trainable params: 0 (0.00 B)

```
[ ]: CNN1.fit(train_generator,
             epochs=50,
             validation_data=test_generator)

# Evaluate
loss, acc = CNN1.evaluate(test_generator)
print(f"\nCNN1 Test Accuracy: {acc:.4f}")
```

```
[6]: # CNN2
CNN2 = Sequential([
    Conv2D(64, (3,3), activation='relu', padding='same',
    ↪input_shape=input_shape),
    BatchNormalization(),
    Conv2D(64, (3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),

    Conv2D(128, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    Conv2D(128, (3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),

    Conv2D(256, (3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.4),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(n_classes, activation='softmax')
])

CNN2.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

CNN2.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_15 (Conv2D)	(None, 128, 128, 64)	640

batch_normalization (BatchNormalization)	(None, 128, 128, 64)	256
conv2d_16 (Conv2D)	(None, 128, 128, 64)	36,928
max_pooling2d_15 (MaxPooling2D)	(None, 64, 64, 64)	0
conv2d_17 (Conv2D)	(None, 64, 64, 128)	73,856
batch_normalization_1 (BatchNormalization)	(None, 64, 64, 128)	512
conv2d_18 (Conv2D)	(None, 64, 64, 128)	147,584
max_pooling2d_16 (MaxPooling2D)	(None, 32, 32, 128)	0
conv2d_19 (Conv2D)	(None, 32, 32, 256)	295,168
max_pooling2d_17 (MaxPooling2D)	(None, 16, 16, 256)	0
flatten_3 (Flatten)	(None, 65536)	0
dense_9 (Dense)	(None, 256)	16,777,472
dropout_3 (Dropout)	(None, 256)	0
dense_10 (Dense)	(None, 128)	32,896
dropout_4 (Dropout)	(None, 128)	0
dense_11 (Dense)	(None, 4)	516

Total params: 17,365,828 (66.25 MB)

Trainable params: 17,365,444 (66.24 MB)

Non-trainable params: 384 (1.50 KB)

```
[ ]: CNN2.fit(train_generator,
             epochs=50,
             validation_data=test_generator)

# Evaluate
```

```
loss, acc = CNN2.evaluate(test_generator)
print(f"\nCNN2 Test Accuracy: {acc:.4f}")
```

```
[7]: # CNN3
CNN3 = Sequential([
    Conv2D(32, (5,5), activation='relu', padding='same',
    ↪input_shape=input_shape),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),

    Conv2D(128, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Conv2D(128, (3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(n_classes, activation='softmax')
])

CNN3.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

CNN3.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_20 (Conv2D)	(None, 128, 128, 32)	832
max_pooling2d_18 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_21 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_19 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_22 (Conv2D)	(None, 32, 32, 128)	73,856

batch_normalization_2 (BatchNormalization)	(None, 32, 32, 128)	512
max_pooling2d_20 (MaxPooling2D)	(None, 16, 16, 128)	0
conv2d_23 (Conv2D)	(None, 16, 16, 128)	147,584
max_pooling2d_21 (MaxPooling2D)	(None, 8, 8, 128)	0
flatten_4 (Flatten)	(None, 8192)	0
dense_12 (Dense)	(None, 128)	1,048,704
dropout_5 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 4)	516

Total params: 1,290,500 (4.92 MB)

Trainable params: 1,290,244 (4.92 MB)

Non-trainable params: 256 (1.00 KB)

```
[ ]: CNN3.fit(train_generator,
            epochs=50,
            validation_data=test_generator)

# Evaluate
loss, acc = CNN3.evaluate(test_generator)
print(f"\nCNN3 Test Accuracy: {acc:.4f}")
```

```
[8]: #To freeze the first-level CNN models so that during the second-level training:
      #ONLY the MLP_meta (second-level model) learns. The CNNs do NOT get updated,
      ↪ again.
      #This is the correct stacking behavior.

      for base_model in [CNN1, CNN2, CNN3]:
          base_model.trainable = False

      # Shared input
      stack_input = Input(shape=input_shape)

      # Get softmax outputs from each base model
```

```

pred1 = CNN1(stack_input)
pred2 = CNN2(stack_input)
pred3 = CNN3(stack_input)

# Concatenate predictions -> meta-features
merged = Concatenate()([pred1, pred2, pred3]) # shape: (None, 3 * n_classes)

meta_input_dim = 3 * n_classes # e.g., 3 * 4 = 12

MLP_meta = Sequential(name="MLP_meta")
MLP_meta.add(Dense(32, activation='relu', input_shape=(meta_input_dim,)))
MLP_meta.add(Dense(32, activation='relu'))
MLP_meta.add(Dropout(0.5))
MLP_meta.add(Dense(n_classes, activation='softmax'))

# Pass merged predictions to MLP_meta
stack_output = MLP_meta(merged)

# Final stacking model
Stacking_Model = Model(inputs=stack_input, outputs=stack_output)

Stacking_Model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

Stacking_Model.summary()

```

Model: "functional_51"

Layer (type)	Output Shape	Param #	Connected to
input_layer_5 (InputLayer)	(None, 128, 128, 1)	0	-
sequential_2 (Sequential)	(None, 4)	1,027,716	input_layer_5[0]...
sequential_3 (Sequential)	(None, 4)	17,365,828	input_layer_5[0]...
sequential_4 (Sequential)	(None, 4)	1,290,500	input_layer_5[0]...
concatenate	(None, 12)	0	sequential_2[0][...

(Concatenate)			sequential_3[0] [...] sequential_4[0] [...]
MLP_meta (Sequential)	(None, 4)	1,604	concatenate[0][0]

Total params: 19,685,648 (75.09 MB)

Trainable params: 1,604 (6.27 KB)

Non-trainable params: 19,684,044 (75.09 MB)

```
[ ]: # Train Stacking Model
      # Now train the stacking ensemble using the generators
      epochs_stack = 3

      history_stack = Stacking_Model.fit(
          train_generator,
          epochs=epochs_stack,
          validation_data=test_generator
      )

      # Evaluate
      loss, acc = Stacking_Model.evaluate(test_generator)
      print(f"\nStacking Model Test Accuracy: {acc:.4f}")
```